



Big Data Systems Foundations

University of Virginia

DS 7200: Distributed Computing

Last Updated: August 26, 2026

SOURCES

Solution Architect's Handbook. 2nd ed. Saurabh Shrivastava and Neelanjali Srivastav.

OBJECTIVES

- Introduce some big data system building blocks
- Provide high-level description of big data systems

CONCEPTS

- server
- client
- database
- data warehouse
- data lake
- domain name system (DNS)
- front end vs back end
- scaling
- streaming vs batch
- architecture diagrams
- horizontal scaling
- vertical scaling
- three-tier architecture
- data ingestion

Introducing System Components

Here we briefly describe components that will come up throughout the course:

server: hardware or software providing a service to another computer program and its user, also known as the *client*. oftentimes, it is a beefy machine. examples: database server, web server.

client: a **user program** that connects to a server to access a service. an example is a web browser.

database: **organized collection of data**, typically stored electronically in a computer system. can be divided into relational and non-relational databases.

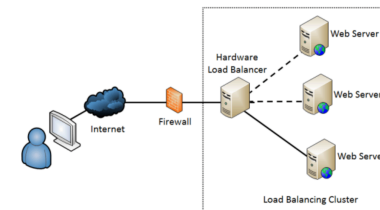
data warehouse: central data repository for storing **structured data**. this is generally a collection of relational databases across an org.

data lake: central data repository for **storing data of any structure**. the benefit is that unstructured and semi-structured data can be stored.

load balancer: device to distribute a set of tasks over a set of resources (generally servers), to enable scale.

for example if a web server has traffic that overwhelms a server, a load balancer can be put in front of a set of servers.

the load balancer will send uniformly distributed data to the servers.



front end: layer of a system that focuses on **visual aspects**, like the web site. typically UX people work on this part.

back end: part of the system focused on structure, business logic and data. for example, the database resides in this part.

cloud solutions: products and services offered by a *cloud provider* which offers pay-as-you-go pricing using their hardware and software.

cloud provider: these companies maintain the hardware and software for customers to use. Currently, the most popular providers are:

- Amazon Web Services (AWS)
- Microsoft Azure

- Google Cloud Platform (GCP)

This course will focus on AWS services.

Domain Name System (DNS)

DNS gets its own subsection because it's so critical and is often the reason for large system failures.

DNS is a mapping service from domain names to IP addresses so your computer knows where to connect.

How it Works

- 1 | You type `www.virginia.edu` into a browser
- 2 | Your computer requests that the DNS server look up the website ip address
- 3 | The DNS server responds with the IP address `132.148.77.44`
- 4 | Your browser connects to that IP address and loads the site

The DNS is globally distributed (spread around a network) and cached (retaining a previously computed result)

The biggest public DNS services operate as huge distributed networks, with servers in many data centers around the world.

The DNS updates dynamically as infrastructure changes, but changes can be manual.

There are many things that can go wrong with DNS, which can lead to systems breaking. This includes:

- Wrong IP
- Deleted record
- Circular references

Scaling

When more computing resources are needed, two approaches are *horizontal scaling* and *vertical scaling*.

vertical scaling: adding resources to the server, such as CPUs, RAM, DISK.

- can be relatively expensive
- there are hardware limits to this scaling
- the server is a single point of failure - if it goes down, the application goes down with it

horizontal scaling: adding more servers

- cloud computing services make this easy
- relatively inexpensive way to scale

- extra servers provide redundancy and avoid single point of failure

Types of Processing

A **batch job** is a job run on a finite amount of data. This might consist of computing analytics or making a set of predictions.

A **streaming job** runs on data that doesn't end. An example is processing data from an X feed.

We will examine each of these jobs in detail. Streaming is harder, since infinite data brings challenges such as what to save and when to report results.

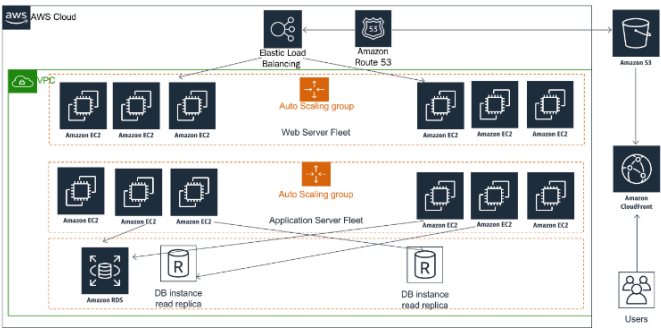
Architecture Diagrams

Architecture diagrams illustrate a system at a high level for ease of understanding.

They are typically created by solution architects and it is helpful to have familiarity.

Below is an example of a *three-tier architecture*, which is very popular. The tiers consist of:

1. design layer: front-end web site
2. application layer (contains things like business logic, machine learning models)
3. data layer: the database



source: *Solution Architect's Handbook*

The tiles represent components specific to Amazon Web Services (AWS), so let's unpack them:

- EC2 comprises servers or *instances* as they call them. There is a set or fleet of instances to support the web layer and a separate set to support the application layer. These fleets

can be scaled (removing or adding instances) as the job size changes; this type of scaling is an example of horizontal scaling.

- Amazon RDS is a relational database service. It is good practice to maintain one or more backups for redundancy. Read replicas are common.
- Amazon Route 53 is a DNS service that takes a web address like `www.yahoo.com` and maps it via a lookup table to its associated ip address.
- Amazon S3 (simple storage service) is a data lake. Specifically it is an *object store* based on key-value pairs of data. This is a NoSQL database. It is very common to store all data in S3.

We will look at other kinds of system architectures later in the course.

Big Data System Process

At a high level, the steps are:

- Data Ingestion
- Data Storage
- Processing
- Reporting & Visualization

Given massive amounts of data, each of these steps are challenging to implement.

Specialized software and hardware were invented to address some of the challenges.

We briefly discuss each of these steps.

Data Ingestion

Data ingestion consists of collecting data for transfer and storage.

Data is typically ingested from databases, streams, logs or files.

Specific tools for ingesting streaming data include Apache Kafka and Amazon Kinesis.

Data comes from devices (sensors, phones, wearables), clickstream logs, server logs and images among other things.

Data types include flat files, images, video and audio.

Data Storage

Choices for data storage will depend on factors including the structure of the data and how it will be consumed.

Most data at rest currently lives in a relational database (consisting of tables that can be joined with common fields).

Semi-structured data (e.g., JSON) and unstructured data are often stored in NoSQL databases.

JSON is very popular for passing data between applications (APIs) due to its simple, hierarchical structure.

It often makes sense to combine different storage solutions to balance cost and latency (how fast the data must be delivered).

Processing

This is where tasks including analytics and predictive modeling take place.

The results may be reported, visualized or stored.

For big data processing, Spark is an excellent tool. It does in-memory processing, using RAM of each machine or *node*.

We will spend much of the course working with Spark, in particular:

- analytical processing
- querying data with Spark SQL
- machine learning with Spark MLlib
- stream processing with Spark Streaming

Reporting & Visualization

This is often the last step: extracting insights and value from data.

Popular tools include:

- Tableau (for beautiful, interactive visuals)
- Power BI (from Microsoft)
- Kibana (open-source tool used for stream data visualization and logs)

Next Steps

Much more can be said on these topics. We will start to dive in and see how these ideas are applied.